ADNI Knowledge Graph Design Blueprint

From Labeled Property Graph to Semantic Knowledge Graph
A Step-by-Step Master's Thesis Implementation Guide

Authors: O. Gungor, S. N. Turhan, O. Pinarer, S. Arib, H. Baazaoui, R. Bouhamoum
Institutions: Galatasaray University · CY Cergy Paris University (ETIS Lab)
Dataset: ADNI (108 tables, ~5,800 columns, ~407K nodes, ~1.16M relationships)
Tools: Neo4j, Elasticsearch, Redis, Python
Target: Semantic KG with Causal Discovery Overlay

Contents

1. LPG vs Knowledge Graph — The Core Distinction

Your current Neo4j implementation is a **Labeled Property Graph (LPG)**. To become a true **Knowledge Graph (KG)**, it needs *semantic grounding* — nodes and relationships must carry machine-interpretable references to standard ontologies, enabling reasoning and inference.

Aspect	LPG (Current State)	True KG (Target State)
Schema	Implicit, application-defined	Formal OWL/RDF ontology references
Relationships	Typed strings (syntactic)	Ontology-grounded (semantic URIs)
Inference	Not supported	IS_A reasoning, transitive closure
Interoperability	System-specific	SNOMED-CT, LOINC, UBERON, HPO
Query Language	Cypher only	Cypher + semantic queries via MAPS_TO
Example Edge	HAS_DIAGNOSIS (string)	ro:RO_0000091 (has_disposition)

Key Insight: You do NOT need to switch to RDF/SPARQL. Neo4j can serve as a KG by **adding ontology properties to existing nodes and relationships**. This is the approach validated by Spasov et al. (2024) with AD-DPC ontology in Neo4j, and by Romano et al. (2024) with AlzKB's OWL2 ontology mapped to Neo4j.

Figure 1: LPG vs Knowledge Graph — Side-by-Side Comparison

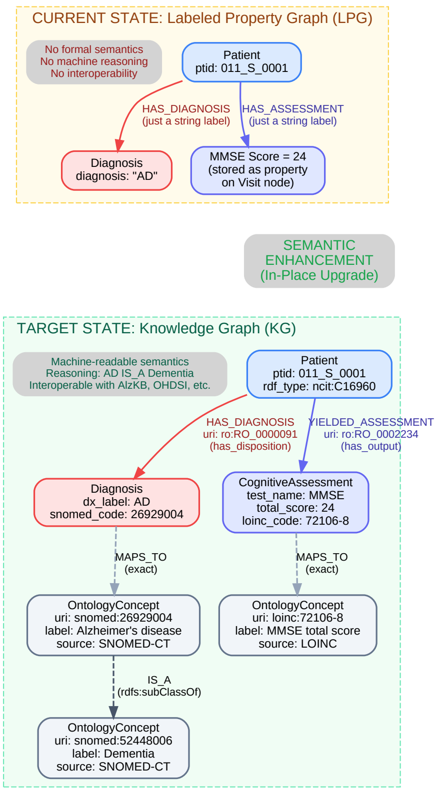

Figure 1: In the LPG (left), relationships are plain string labels. In the KG (right), every entity has ontology type references, and relationships carry Relation Ontology URIs. OntologyConcept nodes form an IS_A hierarchy enabling machine reasoning.

2. Target KG Schema — Complete Node & Relationship Diagram

The following diagram shows the complete target Knowledge Graph schema. Patient remains the hub node. Time data is stored as properties on Visit nodes (not as separate temporal nodes). The design supports incremental insertion of new data. Dashed lines represent MAPS_TO ontology connections. Solid lines are data relationships.

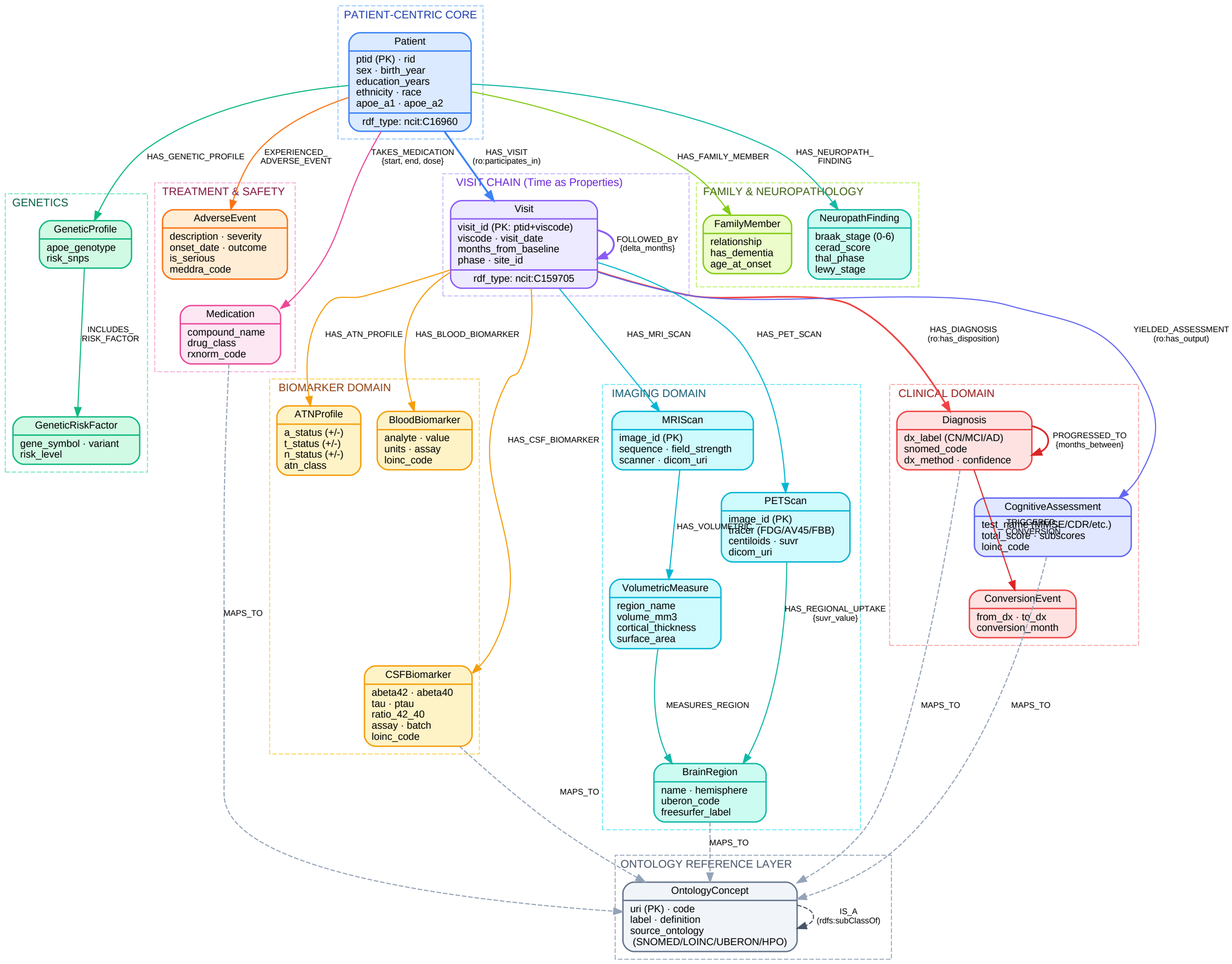

Figure 2: Complete Target KG Schema — Patient-centric design with 17 node types, 20+ relationship types, and an ontology reference layer. All 108 ADNI tables map to these nodes.

Design Principle — Simplicity for Master's Level: Time is stored as properties on Visit nodes (visit_date, months_from_baseline) and as delta_months on FOLLOWED_BY edges. No separate temporal nodes. This is sufficient for causal discovery on baseline-only data and for tracking patient trajectories.

3. Node Catalog with Properties & Indexes

Node Label	Key Properties	Ontology	Source Tables	Index
Patient	ptid (PK), rid, sex, birth_year, education_years, ethnicity, race, apoe_a1, apoe_a2	ncit:C16960	PTDEMOG, APOERES, Study_Entry	Unique: ptid Index: rid
Visit	visit_id (PK: ptid+viscode), viscode, visit_date, months_from_baseline, phase, site_id	ncit:C159705	All tables (join key)	Composite: (ptid, viscode)
Diagnosis	dx_label (CN/MCI/AD), snomed_code, dx_method, confidence	SNOMED-CT: 26929004 (AD) 386806002 (MCI)	DXSUM, BLCHANGE	Index: dx_label, snomed_code
ConversionEvent	from_dx, to_dx, conversion_month	Custom: adni:ConversionEvent	Derived from DXSUM	Index: from_dx
CognitiveAssessment	test_name, total_score, subscores (JSON), loinc_code	LOINC: 72106-8 (MMSE), etc.	MMSE, CDR, ADAS, MOCA, FAQ, etc. (14 tables)	Index: test_name, loinc_code
CSFBiomarker	abeta42, abeta40, tau, ptau, ratio_42_40, assay, batch, loinc_code	LOINC: 72333-6 (Abeta42)	UPENNBIOMK, BIOMARK, FOXLABBSI	Index: assay
BloodBiomarker	analyte, value, units, assay, loinc_code	LOINC per analyte	LABDATA, URMC_LABDATA, FNIHBC, JANSSEN	Index: analyte
ATNProfile	a_status, t_status, n_status, atn_class	Custom: NIA-AA Framework	Derived from CSF+PET+MRI	Index: atn_class
MRIScan	image_id (PK), sequence, field_strength, scanner, dicom_uri	DICOM + SNOMED imaging terms	Key_MRI, MRI3META, Structural_MRI_Images	Unique: image_id
PETScan	image_id (PK), tracer, centiloids, suvr, dicom_uri	SNOMED: 82918005	Key_PET, AV45META, AMYMETA, TAUMETA	Unique: image_id Index: tracer
VolumetricMeasure	region_name, volume_mm3, cortical_thickness, surface_area, hemisphere	UBERON codes per region	UCSFFSX7, UCBERKELEY_*, UCD_WMH	Index: region_name
BrainRegion	name, hemisphere, uberon_code, freesurfer_label	UBERON: 0002421 (Hippocampus)	Reference data (static)	Unique: name+hemisphere
GeneticProfile	apoe_genotype, risk_snps	Gene Ontology / NCBI Gene	APOERES, GENETIC	Index: apoe_genotype
Medication	compound_name, drug_class, rxnorm_code	RxNorm / ATC	BACKMEDS, RECCMEDS, ANTIAMYTX	Index: compound_name
AdverseEvent	description, severity, onset_date, outcome, is_serious, meddra_code	MedDRA codes	ADVERSE, ADSXLIST, RECADV	Index: severity
FamilyMember	relationship, has_dementia, age_at_onset	HPO: HP:0002354	FAMHXPAR, FAMHXSIB, FHQ	Index: relationship

Node Label	Key Properties	Ontology	Source Tables	Index
NeuropathFinding	braak_stage (0-6), cerad_score, thal_phase, lewy_stage	SNOMED neuropath codes	NEUROPATH (164 columns)	Index: braak_stage
OntologyConcept	uri (PK), code, label, source_ontology, definition	Self-referencing (IS_A hierarchy)	Imported from standard ontologies	Unique: uri Index: code

4. Relationship Catalog

Relationship Type	Source -> Target	Properties	Ontology URI
HAS_VISIT	Patient -> Visit	—	ro:RO_0000056 (participates_in)
FOLLOWED_BY	Visit -> Visit	delta_months	time:intervalBefore
HAS_DIAGNOSIS	Visit -> Diagnosis	source_table	ro:RO_0000091 (has_disposition)
PROGRESSED_TO	Diagnosis -> Diagnosis	months_between	ro:RO_0002263 (inverse)
TRIGGERED_CONVERSION	Diagnosis -> ConversionEvent	—	Custom
YIELDED_ASSESSMENT	Visit -> CognitiveAssessment	—	ro:RO_0002234 (has_output)
HAS_CSF_BIOMARKER	Visit -> CSFBiomarker	—	ro:RO_0002234
HAS_BLOOD_BIOMARKER	Visit -> BloodBiomarker	—	ro:RO_0002234
HAS_ATN_PROFILE	Visit -> ATNProfile	—	adni:derived_from
HAS_MRI_SCAN	Visit -> MRIScan	—	ro:RO_0002234
HAS_PET_SCAN	Visit -> PETScan	—	ro:RO_0002234
HAS_VOLUMETRIC	MRIScan -> VolumetricMeasure	—	ro:RO_0002234
MEASURES_REGION	VolumetricMeasure -> BrainRegion	—	bfo:BFO_0000066
HAS_REGIONAL_UPTAKE	PETScan -> BrainRegion	suvr_value	Custom
HAS_GENETIC_PROFILE	Patient -> GeneticProfile	—	ro:RO_0000053
INCLUDES_RISK_FACTOR	GeneticProfile -> GeneticRiskFactor	—	ro:RO_0000053
TAKES_MEDICATION	Patient -> Medication	start, end, dose	ro:RO_0000056
EXPERIENCED_AE	Patient -> AdverseEvent	—	Custom
HAS_FAMILY_MEMBER	Patient -> FamilyMember	—	ro:RO_0000053
HAS_NEUROPATH	Patient -> NeuropathFinding	—	ro:RO_0000053
MAPS_TO	Any -> OntologyConcept	mapping_confidence	skos:exactMatch
IS_A	OntologyConcept -> OntologyConcept	—	rdfs:subClassOf
CAUSES (Phase 2)	Any -> Any	algorithm, p_value, effect_size	ro:RO_0002411 (causally_upstream_of)

5. Index & Constraint Strategy

Uniqueness Constraints (enforce data integrity):

```
CREATE CONSTRAINT patient_ptid FOR (p:Patient) REQUIRE p.ptid IS UNIQUE;
CREATE CONSTRAINT visit_id FOR (v:Visit) REQUIRE v.visit_id IS UNIQUE;
CREATE CONSTRAINT mri_image_id FOR (m:MRIScan) REQUIRE m.image_id IS UNIQUE;
CREATE CONSTRAINT pet_image_id FOR (p:PETScan) REQUIRE p.image_id IS UNIQUE;
CREATE CONSTRAINT brain_region FOR (b:BrainRegion)
REQUIRE (b.name, b.hemisphere) IS UNIQUE;
CREATE CONSTRAINT ontology_uri FOR (o:OntologyConcept) REQUIRE o.uri IS UNIQUE;
```

Performance Indexes (for query anchors):

```
CREATE INDEX patient_rid FOR (p:Patient) ON (p.rid);
CREATE INDEX visit_ptid FOR (v:Visit) ON (v.ptid);
CREATE INDEX visit_viscode FOR (v:Visit) ON (v.viscode);
CREATE INDEX visit_phase FOR (v:Visit) ON (v.phase);
CREATE INDEX dx_label FOR (d:Diagnosis) ON (d.dx_label);
CREATE INDEX dx_snomed FOR (d:Diagnosis) ON (d.snomed_code);
CREATE INDEX assess_test FOR (c:CognitiveAssessment) ON (c.test_name);
CREATE INDEX assess_loinc FOR (c:CognitiveAssessment) ON (c.loinc_code);
CREATE INDEX atn_class FOR (a:ATNProfile) ON (a.atn_class);
CREATE INDEX pet_tracer FOR (p:PETScan) ON (p.tracer);
CREATE INDEX vol_region FOR (v:VolumetricMeasure) ON (v.region_name);
CREATE INDEX med_name FOR (m:Medication) ON (m.compound_name);
CREATE INDEX gene_sym FOR (g:GeneticRiskFactor) ON (g.gene_symbol);
CREATE INDEX ontology_code FOR (o:OntologyConcept) ON (o.code);
CREATE INDEX ontology_src FOR (o:OntologyConcept) ON (o.source_ontology);
```

Indexing Rule of Thumb: Index every property you MATCH/WHERE on. The most common query pattern is: find Patient -> traverse Visits -> filter by diagnosis/biomarker. So Patient.ptid, Visit.ptid, Visit.viscode, and Diagnosis.dx_label are the most critical.

6. How the Research Community Solved This

Project / Authors	Approach	What We Take From It
AD-DPC + Neo4j KG Spasov et al. (2024) Lazarova et al. (2023)	Built a data-independent AD ontology (AD-DPC), populated Neo4j KG with ADNI data (16,227 entries, 2,404 participants).	Closest precedent. Proved Neo4j can be semantic KG without RDF. We adopt this but at 108 tables vs their ~12.
AlzKB Romano et al. (2024)	Largest AD-specific KG: 118,902 entities, 1.3M relationships. OWL2 ontology in Protege, deployed in Neo4j via 21 databases.	Our integration target. Their ontology classes provide ready-made vocabulary for MAPS_TO edges.
ADKG Yang et al. (2025)	GPT-4 augmented KG from 160K PubMed abstracts. SpERT for NER. Entity linking to NCBI Gene, ChEBI, DrugBank, HPO.	Their entity linking pipeline (simstring + multi-database) is gold standard for resolving ADNI columns to ontology terms.
Causal Discovery on ADNI Shen et al. (2020)	Tested FCI and FGES on ADNI biomarkers. FCI with expert guidance recovered canonical A->T->N cascade. FGES: 71% precision.	Direct validation that our FCI approach works on ADNI data. Unguided algorithms miss biology -- we use KG ontology as priors.
Unified Framework Dobreva et al. (2025)	Systematic review of 43 AD KG papers. Proposed AD-KG 2.0 framework with decision tree.	Validates our Construct -> Discover -> Represent methodology. Recommends structured data KGs before literature-derived.

What's Original: No existing work combines (1) a full-scale ADNI data graph (108 tables, 407K nodes) with (2) semantic ontology grounding AND (3) causal discovery integration where discovered causal edges become native KG relationships. Spasov had ontology but limited data; AlzKB has scale but no patient-level data; Shen had causal discovery but no KG. This thesis unifies all three.

7. Step-by-Step Implementation Plan

Phase 1: Schema Migration (LPG -> KG)

Step 1: Add ontology properties to existing nodes (in-place upgrade). Run Cypher SET commands to add snomed_code, loinc_code, uberon_code properties. Don't rebuild — transform in place.

Step 2: Create OntologyConcept nodes (reference layer). Import ~200-300 curated concepts relevant to AD from SNOMED-CT, LOINC, UBERON, HPO. Build IS_A hierarchies.

Step 3: Create MAPS_TO relationships. Connect existing data nodes to their OntologyConcept references. This is what makes the graph 'semantic.'

Step 4: Apply uniqueness constraints and indexes. All constraints from Section 5, before inserting new data.

Phase 2: Causal Discovery Preparation

Step 5: Extract flat feature matrix from KG for causal algorithms. Cypher query pulls baseline-visit data (MMSE, CDR, Abeta42, tau, ptau, APOE, ATN) into a tabular format for PC/FCI/GES/DAG-GNN.

Step 6: Run causal discovery algorithms in parallel. FCI is prioritized for handling latent confounders in medical data. Compare outputs per Shen et al. (2020) methodology.

Step 7: Embed causal edges back into the KG. Discovered causal relationships become native CAUSES edges with metadata (algorithm, p_value, effect_size, ro:RO_0002411 URI).

Phase 3: Validation & Integration

Step 8: Validate causal edges against AlzKB and literature. Cross-reference discovered edges with AlzKB's existing relationships. Mark as literature-validated if confirmed.

Step 9: Run DoWhy for causal inference. Use KG-derived causal DAG as input. Estimate specific effects (e.g., 'causal effect of amyloid positivity on MMSE decline'). Run refutation tests.

Step 10: Document and defend. Write up Construct -> Discover -> Represent pipeline. Show before/after: LPG query vs KG semantic query vs causal query.

8. Incremental Data Ingestion Pattern

The graph must accept new data without rebuilding. The key pattern is **MERGE with ON CREATE / ON MATCH**, which provides idempotent upsert behavior:

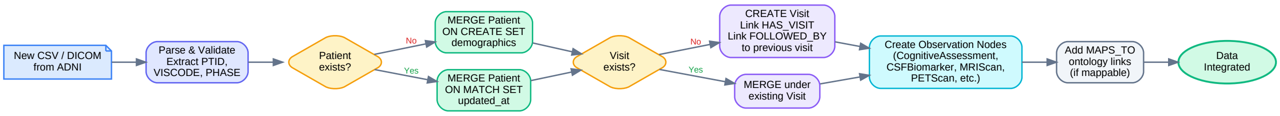

Figure 3: Incremental data ingestion — MERGE-based upsert pattern ensures idempotent inserts. Every new CSV row or DICOM file follows this pipeline.

```
def ingest_new_data(driver, table_name, rows):
    with driver.session() as session:
        for row in rows:
            ptid = row.get('PTID')
            viscode = row.get('VISCODE2') or row.get('VISCODE')
            visit_id = f"{ptid}_{viscode}"
            # 1. MERGE Patient
            session.run("""
                MERGE (p:Patient {ptid: $ptid})
                ON CREATE SET p.created_at = datetime()
                ON MATCH SET p.updated_at = datetime()
            """, ptid=ptid)
            # 2. MERGE Visit + link
            session.run("""
                MATCH (p:Patient {ptid: $ptid})
                MERGE (v:Visit {visit_id: $vid})
                ON CREATE SET v.viscode=$vs, v.phase=$ph
                MERGE (p)-[:HAS_VISIT]->(v)
            """, ptid=ptid, vid=visit_id, vs=viscode,
                 ph=row.get('PHASE'))
            # 3. Create domain-specific observation
            create_observation(session, table_name, visit_id, row)
```

9. AlzKB Integration Strategy

AlzKB (Romano et al., 2024) contains 118,902 entities and 1,309,527 relationships from 21 external databases. It is literature-derived knowledge. Your graph has patient-level clinical data. The integration creates a bridge between *what is known about AD* (AlzKB) and *what is observed in patients* (your KG).

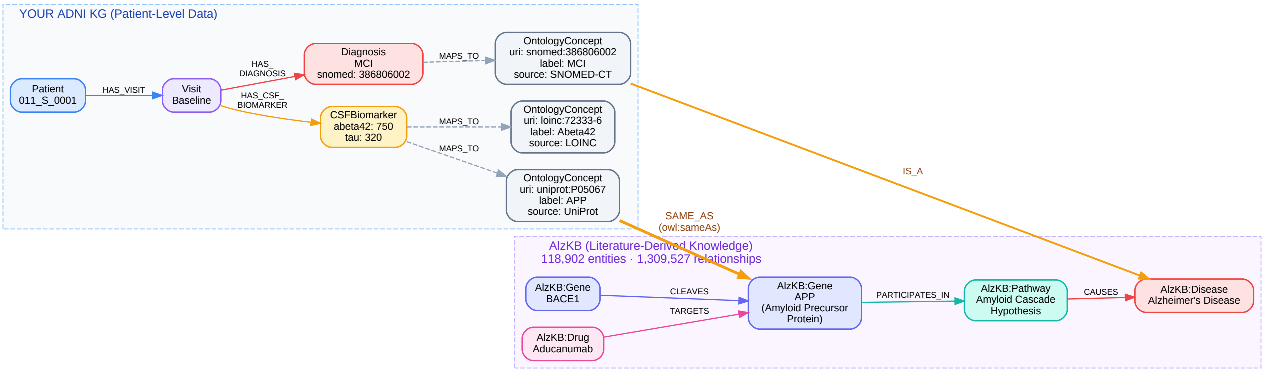

Figure 4: Bridge pattern — OntologyConcept nodes in your KG link to AlzKB entities via SAME_AS (owl:sameAs) relationships. You import only the ~200-300 overlapping concepts, not all 118K AlzKB entities.

Practical Approach (Master's Level): Import only concepts overlapping with your ADNI data: AD-relevant genes (APOE, APP, PSEN1, MAPT), biomarker analytes (Abeta42, tau, ptau, NfL), brain regions (hippocampus, entorhinal cortex), and drugs your patients take. Create SAME_AS links between your OntologyConcept nodes and AlzKB equivalents.

10. Mapping from headers.json to KG Nodes

All 108 ADNI tables in headers.json map to the KG node types. Each table's PTID+VISCODE join pattern anchors data to the correct Patient -> Visit path.

KG Node	ADNI Tables	Key Columns
Patient	PTDEMOG, APOERES, Study_Entry, ARM	PTID, RID, PTGENDER, PTDOB, PTEDUCAT, APGEN1, APGEN2
Visit	All 108 tables (join key)	PTID, VISCODE/VISCODE2, VISDATE, PHASE, SITEID
Diagnosis	DXSUM, BLCHANGE	DXCURREN, DXCHANGE, DXCONV, DIAGNOSIS
CognitiveAssessment	MMSE, CDR, ADAS, MOCA, FAQ, GDSCALE, NPI/NPIQ, NEUROBAT, ECOGPT/SP, STAIAD (14 tables)	TOTSCORE, TOTAL13, CDGLOBAL, MMSCORE, etc.
CSFBiomarker	UPENNBIOMK_ROCHE_ELECSYS, BIOMARK, FOXLABBSI	ABETA42, ABETA40, TAU, PTAU, BATCH
BloodBiomarker	LABDATA, URMC_LABDATA, FNIHBC, JANSSEN	TestName, ResultValue, Units, Flags
MRIScan	Key_MRI, Structural_MRI_Images, MRI3META, MRIMETA, MRI_Images_with_AI (7 tables)	image_id, modality, field_strength, SCANDATE
PETScan	Key_PET, PET_Images, AV45META, AMYMETA, TAUMETA (8 tables)	image_id, RADTRACER, CENTILOIDS, SCANDATE
VolumetricMeasure	UCSFFSX7, UCBERKELEY_AMY_6MM, UCBERKELEY_TAU_*, UCD_WMH, DTIROI_* (7 tables)	ST* columns (region volumes/thickness/area)
GeneticProfile	APOERES, GENETIC	APGEN1, APGEN2, SNP columns
Medication	BACKMEDS, RECCMEDS, ANTIAMYTX	Compound, drug class columns
AdverseEvent	ADVERSE, ADSXLIST, RECADV	AECOMPOUND, AEHSEVR, AEOUTCOME, AX* symptoms
FamilyMember	FAMHXPAR, FAMHXSIB, FHQ, RECFHQ	Relationship, dementia history
NeuropathFinding	NEUROPATH (164 columns)	Braak, CERAD, Thal, Lewy, TDP-43

Completeness: This mapping covers all 108 tables. Study/Admin tables (CONSENTS, BLSCHECK, CLIELG, EXCLUSIO, etc.) inform Visit and Patient properties rather than creating separate nodes. Other Assessment tables (AMAS, CCI, CSSRSAD, etc.) map to CognitiveAssessment with the test_name property distinguishing each instrument.